



---

# Universidad de Valladolid

Escuela Técnica Superior de Ingenieros de  
Telecomunicación

Configuración y actualización dinámica de modelos  
de aprendizaje profundo

GRADO EN INGENIERÍA DE TECNOLOGÍAS DE LA TELECOMUNICACIÓN

## TRABAJO DE FIN DE GRADO

Autor:  
Andrés García Treviño

Tutor:  
Dr. D. Pablo Casaseca de la Higuera

Valladolid, junio 2026

# Resumen

El crecimiento exponencial del número de dispositivos capaces de generar tráfico de red ha aumentado significativamente la necesidad de organizar los recursos disponibles para asegurar un nivel estable de calidad de servicio. La predicción del tráfico de red se ha convertido en la mayor aliada para evitar congestiones y pérdidas de servicio. Sin embargo, los modelos predictivos tradicionales tienen muchas dificultades para seguir el ritmo de cambio del tráfico de red actual. Como el tráfico de red puede ser modelado como una secuencia temporal, en este trabajo se estudian y comparan varios modelos de redes neuronales de aprendizaje profundo aplicadas a la predicción de series temporales con capacidad de adaptarse a las variaciones y anomalías mediante entrenamiento incremental. Se han estudiado varias arquitecturas que combinan redes convolucionales con redes recurrentes LSTM sobre el dataset Milan, que recoge el tráfico de red de la ciudad de Milán, Italia, entre noviembre y diciembre de 2013.

Los resultados de los modelos, analizados a través de diferentes métricas estadísticas, demuestran que los modelos que integran capas convolucionales y *online learning* son ampliamente superiores a los modelos tradicionales, incluso en situaciones de cambios bruscos en el patrón de los datos. El modelo ideal, presentando una capa CNN y una capa LSTM llega a reducir el error (NRMSE) cometido en las predicciones desde un 0.1778 conseguido con un modelo neuronal LSTM sencillo hasta un 0.142. Por último, los modelos analizados en este trabajo han sido optimizados al máximo para conseguir el mayor rendimiento posible en cuanto a carga computacional y tiempo de ejecución.

## Palabras clave

*Online learning*, predicción, redes neuronales, redes convolucionales, tráfico de red.

## Abstract

The exponential growth of telecommunication devices has raised the need of correctly allocating the network resources to ensure a reliable Quality of Service level. Network traffic prediction has become the greatest ally in the battle against network congestion and loss of service. Nonetheless the traditional prediction models are burdened with great difficulties to follow the changes in network traffic. As it can be seen as a temporal sequence, this work puts several *online deep learning* models under the microscope to elucidate whether they are able to adapt dynamically to changes in the temporal series. Various distinct architectures featuring convolutional and LSTM layers have been tested against the Milan dataset, which holds information about the traffic load in the city of Milan, Italy, between November and December 2013.

The models' outputs have been analysed via different statistical methods, demonstrating the vast superiority of the models combining convolutional and recurrent layers over the more traditional models, even when the pattern abruptly changes. The best model so far, characterized by a single CNN layer connected to an LSTM layer has been able to boost precision, measured as NRMSE, from 0.1778 to 0.142. Last but not least, all the models scrutinized by this work have been successfully optimized to achieve the greatest possible computation speeds.

## Keywords

Convolutional networks, network traffic, neural networks, online learning, prediction.

# Agradecimientos

A mi familia, a mis amigos, a mis profesores y, sobre todo, al Dr. D. Juan Pablo Casaseca de la Higuera. A todos ellos les agradezco su infinita paciencia.

# Índice

|                                                                                                                     |           |
|---------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                                                              | <b>5</b>  |
| 1.1. Objetivos . . . . .                                                                                            | 6         |
| 1.2. Estructura del documento . . . . .                                                                             | 6         |
| <b>2. Fundamentos del <i>Machine Learning</i> y <i>Deep learning</i></b>                                            | <b>7</b>  |
| 2.1. Inteligencia Artificial . . . . .                                                                              | 7         |
| 2.2. <i>Machine Learning</i> . . . . .                                                                              | 7         |
| 2.2.1. Técnicas de entrenamiento . . . . .                                                                          | 8         |
| 2.2.2. <i>Online learning</i> . . . . .                                                                             | 9         |
| 2.3. <i>Deep learning</i> . . . . .                                                                                 | 10        |
| <b>3. Redes neuronales</b>                                                                                          | <b>11</b> |
| 3.1. El <i>Perceptron</i> . . . . .                                                                                 | 11        |
| 3.2. Funciones de activación . . . . .                                                                              | 12        |
| 3.3. Entrenamiento de redes neuronales y <i>backpropagation</i> . . . . .                                           | 13        |
| 3.3.1. Gradient descent . . . . .                                                                                   | 13        |
| 3.3.2. Funciones de coste . . . . .                                                                                 | 14        |
| 3.3.3. <i>Backpropagation</i> . . . . .                                                                             | 15        |
| 3.4. Selección de modelos . . . . .                                                                                 | 16        |
| 3.5. Preprocesado de los datos de entrada . . . . .                                                                 | 17        |
| 3.6. Técnicas de regularización (dropout, gradient clipping, l1, l2, overfitting, vanishing gradients...) . . . . . | 18        |
| <b>4. Predicción de series temporales</b>                                                                           | <b>18</b> |
| 4.1. Modelos autorregresivos de medias móviles . . . . .                                                            | 19        |
| 4.2. Modelos de aprendizaje profundo aplicados a la predicción de series temporales . . . . .                       | 20        |
| 4.2.1. Redes neuronales recurrentes . . . . .                                                                       | 21        |

|           |                                                                                              |           |
|-----------|----------------------------------------------------------------------------------------------|-----------|
| 4.2.2.    | Minibatch con las series temporales . . . . .                                                | 22        |
| 4.2.3.    | Inconvenientes de las neuronas recurrentes . . . . .                                         | 22        |
| 4.2.4.    | LSTM . . . . .                                                                               | 22        |
| <b>5.</b> | <b>Estructuras avanzadas de <i>Deep Learning</i> para la predicción de series temporales</b> | <b>22</b> |
| 5.1.      | CNNs . . . . .                                                                               | 22        |
| 5.1.1.    | Filtros, convoluciones, neuronas, ecuaciones... . . . .                                      | 22        |
| 5.2.      | CNN+LSTM . . . . .                                                                           | 22        |
| 5.2.1.    | Neuronas, conexiones, ecuaciones, flujo de datos... . . . .                                  | 22        |
|           | <b>Referencias</b>                                                                           | <b>23</b> |

## Índice de figuras

## Índice de tablas



# 1. Introducción

Las redes de telecomunicaciones tienen como finalidad el intercambio de información entre un nodo transmisor y un nodo receptor. Es un concepto sencillo afectado por un problema de números. Siguiendo la ley de Bell, el número de dispositivos ha aumentado exponencialmente desde 1 ordenador básico en un departamento de una empresa a múltiples dispositivos por persona (teléfonos móviles, relojes inteligentes...) [1]. Casi con la misma tendencia ha evolucionado la tecnología de las telecomunicaciones, impidiendo que el crecimiento de los dispositivos saturase las capacidades de la red. De esta forma, en la actualidad existe una infinidad de nodos transmisores y receptores.

Los flujos de información intercambiada entre los nodos transmisores y receptores forma el tráfico de red. Dicho tráfico puede ser representado y medido. Las medidas o muestras realizadas en diferentes instantes de tiempo pueden aunarse en una serie temporal. Como todas las series, cabe la posibilidad de que sigan un patrón que pueda utilizarse para predecir medidas futuras a partir del histórico de medidas pasadas [2]. Con la inmensa cantidad de flujos de información existentes a día de hoy, resulta imposible analizarlos mediante la intervención humana. Sin embargo, la situación planteada representa el caso de uso de la inteligencia artificial y *Deep Learning*: averiguar el patrón subyacente de un conjunto de datos [3].

La predicción del tráfico de red tiene numerosísimas aplicaciones como, por ejemplo, la correcta planificación y organización de los recursos de las redes de telecomunicaciones. Esto se traduce en un menor desperdicio de recursos, que a su vez se traduce en una inversión económica más eficiente, sin menoscabar la calidad del servicio proporcionado por la red de telecomunicaciones [4]. También es posible aplicar la predicción de tráfico de red a tareas como la detección de anomalías o intrusos, monitorización o mantenimiento [5].

En la actualidad existen multitud de aplicaciones que generan flujos de datos a demanda. Algunos ejemplos son el *media streaming* o los modelos de negocio *as a Service* (XaaS) [6]. Los modelos de predicción estáticos convencionales no son capaces de adaptarse al dinamismo y la falta de estacionariedad de dichos flujos de datos.

Para solucionar dicho problema se han desarrollado modelos capaces de adaptarse a los cambios en las tendencias o patrones subyacentes en los flujos de tráfico. Estos modelos se basan en *online active deep learning*. Esta técnica permite reentrenar los modelos a medida que se incorporan nuevas muestras para adaptarse a ellas en un proceso llamado *concept drift* [7–9].

Este trabajo se basa en la investigación realizada en [10,11] acerca de modelos LSTM que implementan actualizaciones incrementales donde las muestras sobre las que los modelos tuvieron un mayor error toman mayor importancia. En este trabajo se plantea la hipótesis de que un modelo que combine arquitecturas de redes neuronales convolucionales y de redes neuronales recurrentes tenga una precisión predictiva en *online learning* superior a aquellos modelos basados únicamente en redes neuronales recurrentes.

## 1.1. Objetivos

El principal objetivo de este trabajo es analizar la capacidad de adaptación mediante entrenamiento incremental de diferentes modelos de redes neuronales convolucionales recurrentes ante cambios en el patrón subyacente de series temporales.

Dentro del objetivo principal se establecen los siguientes objetivos secundarios:

- La comprensión y dominio del marco teórico utilizado en este trabajo.
- El manejo de herramientas de desarrollo como Python o Kaggle.
- La mejora del modelo base proporcionado.
- Evaluar el impacto producido por las redes neuronales convolucionales respecto al modelo base.
- Evaluar el rendimiento de los modelos en diferentes cuadrículas de 49 y 225 celdas.

## 1.2. Estructura del documento

Esta memoria se divide en 5 capítulos: introducción, antecedentes y estado del arte, entorno experimental, exposición de resultado y discusión y conclusiones y líneas futuras.

El Capítulo 1 se desarrolla la motivación y los objetivos de este trabajo. Además, se plantean la hipótesis y los objetivos y se añade una breve descripción de la estructura de la memoria.

El Capítulo ?? ofrece una visión general del marco teórico relevante a la predicción de series temporales, el aprendizaje automático, *deep learning*, *online learning* y las diferentes arquitecturas de redes neuronales convolucionales y recurrentes.

El Capítulo ?? presenta la base de datos utilizada, las arquitecturas analizadas y los criterios de selección y evaluación de modelos de aprendizaje automático aplicados en este trabajo.

El Capítulo ?? muestra y analiza el rendimiento de los distintos modelos propuestos. También interpreta los resultados obtenidos y comenta las relaciones presentes entre modelos y resultados.

El Capítulo ?? resume las aportaciones de este trabajo y presenta las conclusiones obtenidas. También propone líneas de actuación futuras.

## 2. Fundamentos del *Machine Learning* y *Deep learning*

### 2.1. Inteligencia Artificial

A lo largo de la historia el ser humano ha tratado de automatizar diversas tareas para obtener una mayor eficiencia en su ejecución, reducir su complejidad o liberar al ser humano de las mismas. La programación tal y como la conocemos ahora tiene el mismo objetivo y tuvo su inicio con Ada Lovelace y su *Analytical Engine* en 1843.

El *Analytical Engine* fue el primer ordenador de uso general conocido aunque se limitaba a repetir un mismo conjunto de órdenes perfectamente definidas. En 1950, un grupo de programadores da forma a lo que hoy conocemos como Inteligencia Artificial: la automatización de tareas que requieren del intelecto humano para ser realizadas y completadas.

Un gran ejemplo de Inteligencia Artificial fue DeepBlue: un programa que, a través del procesamiento de unos datos de entrada y una serie de reglas o patrones definidos que relacionan las entradas con las salidas, era capaz de jugar al ajedrez. DeepBlue no era capaz de realizar ningún tipo de aprendizaje y, aun así, fue una Inteligencia Artificial capaz de superar a grandes maestros mundiales del ajedrez como Gary Kasparov.

Aquellas inteligencias artificiales incapaces de aprender como DeepBlue son clasificadas como inteligencias artificiales simbólicas. Sin embargo, el concepto Inteligencia Artificial es muy amplio y engloba otros conceptos como el *Machine Learning*, que engloba a su vez el *Deep learning*.

### 2.2. *Machine Learning*

Deep Blue fue posible gracias a que sus programadores conocían a la perfección el patrón que relacionaba las entradas, en este caso la posición actual de las figuras de ajedrez en el tablero, y las salidas, siendo éstas últimas los movimientos que DeepBlue debía realizar sobre sus figuras en su turno de juego. Este patrón estaba extremadamente bien definido en las reglas del ajedrez y DeepBlue fue programado manualmente con un conjunto muy extenso de jugadas y decisiones que cubrían todas las posibilidades de juego.

Aun con la eficiencia mostrada por DeepBlue en el ajedrez, estos ejemplos de Inteligencia Artificial resultaron ser pésimos en la ejecución de tareas más complejas y poco definidas como la clasificación de imágenes o el procesamiento de texto.

*Machine Learning*, también llamado aprendizaje automático, le da una vuelta al problema: en vez de calcular las salidas mediante el procesamiento de las entradas y las reglas que relacionan las entradas y las salidas, se procesan las entradas y sus correspondientes salidas para averiguar los patrones que relacionan a ambas. De

esta forma, la Inteligencia Artificial puede embarcarse en la resolución de problemas cuyas reglas no están definidas de forma precisa o son demasiado extensas o arduas de definir.

Para estimar si los patrones calculados son correctos, es necesario evaluarlos midiendo la distancia entre las salidas reales o esperadas correspondientes a ciertas entradas y las salidas obtenidas aplicando las reglas obtenidas a las entradas. La forma en la que se calcula esta distancia está definida en la función de coste o *loss function*. Esta distancia es integrada de nuevo en el algoritmo de *Machine Learning* como retroalimentación para aumentar su precisión a la hora de averiguar las reglas entre las entradas y las salidas en iteraciones sucesivas. Este paso final completa el proceso de aprendizaje o entrenamiento, que finaliza cuando la distancia entre las salidas esperadas y las salidas obtenidas es suficientemente pequeña.

Si definimos los datos de entrada y sus salidas esperadas como  $E$  y a la función de coste como  $R$ , es posible definir el *Machine Learning* o aprendizaje automático: Un programa aprende de la experiencia  $E$  con respecto a una tarea  $T$  y una métrica de rendimiento  $R$  si su rendimiento en la tarea, medido por  $R$ , mejora con más experiencia  $E$ .

### 2.2.1. Técnicas de entrenamiento

Existen multitud de algoritmos de que pueden ser clasificados según su método de entrenamiento en: aprendizaje por refuerzo, aprendizaje no supervisado y aprendizaje supervisado.

El aprendizaje por refuerzo utiliza un sistema de recompensas y penalizaciones para que un agente de aprendizaje automático, mediante prueba y error, ajuste sus decisiones con el fin de maximizar las recompensas obtenidas y minimizar las penalizaciones aplicadas por el sistema.

Es especialmente útil para aprender decisiones secuenciales como el guiado autónomo de un robot por una ruta definida o la optimización de unos recursos como el uso de refrigeración industrial en centros de datos. En ambos casos, el sistema de recompensas asigna una valoración positiva a aquellas decisiones que mantienen al robot en la ruta o estabilizan la temperatura de los servidores mientras evalúa negativamente aquellas decisiones que desvían al robot o llevan la temperatura de los servidores fuera de un rango óptimo establecido.

Algunos ejemplos de algoritmos de aprendizaje por refuerzo son SARSA, Q-Learning...

El aprendizaje no supervisado está pensado para la organización y definición de un conjunto de datos. Los algoritmos englobados en esta técnica de entrenamiento deben encontrar los patrones y estructuras subyacentes en los datos de entrada, sin esperar una salida relacionada con los mismos. El aprendizaje no supervisado busca agrupar los datos en distintos sectores que maximicen la distancia intersector y minimicen la distancia intrasector.

Es especialmente útil en la clasificación de los datos según sus características como la segmentación de clientes, en la detección de anomalías como los fraudes financieros o en la reducción de la dimensionalidad de los datos como la simplificación de bases de datos. Algunos ejemplos de algoritmos de aprendizaje no supervisado son *k-means clustering*, *Support Vector Clustering* o el análisis de componentes principales (PCA).

Finalmente, el aprendizaje supervisado, en donde se enclaustra este trabajo, utiliza pares entrada-salida perfectamente definidos para entrenar sus algoritmos. Estos pares son conocidos como datos etiquetados y tienen la función de actuar como referencia para la medición de la distancia entre la salida obtenida y la salida esperada. Los modelos de aprendizaje supervisado deben, utilizando dichas distancias, ajustar de la forma más precisa posible el patrón que relaciona las entradas y las salidas.

Es especialmente útil en la clasificación como, por ejemplo, la identificación de correos electrónicos de SPAM y la predicción de datos como la predicción de eventos meteorológicos. Algunos ejemplos de algoritmos de aprendizaje supervisado son los árboles de decisión, *Support Vector Machines*, la regresión lineal y no lineal y las neuronas artificiales.

### 2.2.2. *Online learning*

Otra forma de clasificar los algoritmos de aprendizaje automático recae en su capacidad para aprender de forma incremental a medida que el modelo procesa nuevos datos. Los modelos estáticos u *offline* son entrenados con todos los datos disponibles y pierden la capacidad de modificarse a sí mismos una vez puestos en producción. Se convierten en algoritmos que aplican las mismas reglas a todos los nuevos datos que procesen sin excepción.

A medida que el tiempo avanza los datos disponibles cambian debido a la constante evolución de todo lo que nos rodea, y con ellos el patrón subyacente que se desea estimar a través de la inteligencia artificial. Este fenómeno, llamado *data drift* o *concept drift*, no afecta a los modelos estáticos por lo que se tornan cada vez más imprecisos e incapaces de proporcionar un resultado correcto. Por ejemplo, en el hipotético caso de la implementación de una nueva regla en la normativa del ajedrez, la soberanía de DeepBlue desaparecería. El mismo concepto es aplicable a los modelos de aprendizaje automático: un modelo estático de *clustering* entrenado sobre un mercado determinado perdería su eficacia ante un cambio en los patrones de compra o en las necesidades de los consumidores.

Para mitigar este problema, los modelos estáticos deben ser reentrenados de cero con todos los datos disponibles, incluidos los datos afectados por el *data drift* y volver a ser lanzados a producción. Esto requiere una gran cantidad de capacidad de cómputo y en tiempos de espera elevados mientras se reentrena el modelo, lo que se traduce en grandes inversiones económicas en recursos como CPUs, GPUs... y la pérdida de beneficios al no tener un buen modelo en producción durante el reentrenamiento.

Por otra parte, los modelos de aprendizaje incremental, también llamados modelos *online*, son entrenados en numerosas iteraciones sucesivas utilizando un conjunto reducido de datos en cada iteración. De esta forma cada iteración de entrenamiento es rápida y ligera. El modelo se adapta a los nuevos datos introducidos para evitar quedar obsoleto ante el *concept drift*. Esto implica que el proceso de aprendizaje no termina nunca, ni siquiera cuando el modelo es lanzado a producción.

Un aspecto importante del *online learning* es la tasa de aprendizaje. Dependiendo de la frecuencia con la que el patrón subyacente de los datos cambia, los modelos *online* deberán adaptarse en mayor o menor medida a los nuevos datos. Como los recursos *hardware* son limitados, también lo es la capacidad de los modelos. A medida que se adaptan a los nuevos patrones, pierden eficacia con los antiguos en un proceso llamado *catastrophic forgetting*.

### 2.3. *Deep learning*

Con todo esto, los algoritmos de *Machine Learning* son capaces de resolver problemas más complejos que la Inteligencia Artificial simbólica pero es posible ir un paso más allá. Aunque el *Machine Learning* pueda aprender los patrones que relacionan unas entradas con unas salidas, el *Deep learning*, también llamado aprendizaje profundo, utiliza varias capas jerárquicas de aprendizaje para combinar las estructuras más simples obtenidas en las primeras capas en patrones más complejos que representen los datos procesados de forma mucho más precisa.

En *Deep Learning*, los algoritmos utilizan capas de neuronas artificiales organizadas de forma sucesiva. Existe una gran variedad de neuronas artificiales, cada una de ellas especializada en una tarea concreta como aplicar un mismo filtro a un conjunto de datos de entrada, almacenar información relevante sobre los datos o reestructurar y redimensionar los datos procesados.

Al tener múltiples capas de neuronas, los modelos de aprendizaje profundo son capaces de procesar los datos sin ninguna intervención humana, mientras que los algoritmos de *Machine Learning* necesitan de la intervención del ser humano para refinar y adaptar los datos manualmente a las necesidades de cada algoritmo. Sin embargo, los modelos de *Deep Learning* necesitan grandes cantidades de datos para poder ser entrenados, lo que dificulta enormemente la interpretación de los procesos llevados a cabo por las redes neuronales, además de una gran inversión en componentes electrónicos especializados en dichos cálculos como las GPUs (graphical Processing Units). Un ejemplo de esta inversión se encuentra en el reciente giro radical de la producción de GPUs de Nvidia, el mayor fabricante de GPUs a nivel mundial, con el nuevo objetivo de satisfacer las ingentes necesidades del mercado del *Deep Learning*.

Aunque las redes neuronales nacieron en 1943 con el modelo lógico de neurona artificial de McCulloch y Pitts, no fueron rentables hasta el desarrollo de nuevas arquitecturas y funciones de activación, métodos de aprendizaje y ajuste como *back-propagation* y la mejora de los componentes electrónicos disponibles. En la actualidad, los modelos de aprendizaje profundo son los algoritmos predeterminados y

más eficientes en prácticamente todo el área de Inteligencia Artificial, desde visión artificial hasta procesamiento de lenguaje.

En estos últimos años desde la expansión del aprendizaje profundo, se han conseguido hitos como, entre otros, la clasificación de imágenes o el procesamiento de audio y lenguaje a nivel humano, la aparición de asistentes digitales y modelos conversacionales como ChatGPT o Microsoft Copilot o la mejora de sistemas de recomendaciones y búsquedas web.

Este trabajo se enmarca dentro del ámbito del *Deep Learning*, más concretamente en la predicción de tráfico de red a través de redes neuronales profundas. La predicción de series temporales, como es el caso del tráfico de red, es una tarea ardua que conlleva la estimación del patrón existente en la serie. Por lo tanto, cumple con todos los requisitos mencionados anteriormente para ser automatizada mediante algoritmos de aprendizaje profundo.

### 3. Redes neuronales

Las redes neuronales son los modelos de aprendizaje profundo por excelencia. Están formadas por neuronas artificiales que utilizan una serie de pesos para procesar unos datos de entrada. Su primera aparición fue en 1943 como una simple descripción del funcionamiento de las neuronas presentes en el sistema nervioso animal: una neurona con salida binaria activará su salida cuando un cierto número de sus entradas, también binarias, estén activas.

#### 3.1. El *Perceptron*

Aunque las neuronas binarias pueden combinarse para formar redes capaces de procesar expresiones lógicas complejas, no son más que una combinación de puertas lógicas como las presentes en cualquier circuito electrónico. En 1957 Rosenblatt redefinió las neuronas artificiales para eliminar sus restricciones binarias, permitiendo un diseño basado en entradas y salidas continuas. Estas neuronas son llamadas TLU (*Threshold Linear Unit*).

Las TLUs reciben como entradas una serie de valores  $x_j$  donde  $J$  es la cantidad de entradas recibidas. Las TLUs calculan la combinación lineal  $z$  de las entradas con una serie de pesos  $w_j$  asociados a cada entrada y un término independiente  $b$ . Seguidamente, se aplica una función de activación  $\phi(z)$  que acota la salida  $y$  de las TLUs en un rango de valores típicamente comprendido entre -1 y 1 o entre 0 y 1. De esta forma se evita que la salida de las TLUs crezca hasta infinito si se encadenan varias TLUs.

$$z = b + \sum_{j=1}^J w_j * x_j = \mathbf{w}^T \mathbf{x} + b \quad (1)$$

$$y = \phi(z) \quad (2)$$

Una neurona TLU con una función de activación binaria como la Heaviside puede ser utilizada en tareas como la clasificación binaria. Sin embargo, su verdadero poder reside en la agrupación de varias TLUs en una misma capa. Esta es la arquitectura de redes neuronales más básica: el *Perceptron*, capaz de, por ejemplo, clasificar los datos en  $J+1$  categorías siendo  $J$  el número de neuronas TLU presentes en la arquitectura.

Cada TLU está conectada a todas las entradas en una disposición llamada capa completamente conectada o *dense layer*. Esta disposición no está presente en todas las arquitecturas como, por ejemplo, en las neuronas convolucionales. La Figura ?? describe un *Perceptron* de dos entradas y tres TLUs con tres salidas. En este caso, en forma matricial con tantas filas  $i$  como TLUs y tantas columnas  $j$  como entradas, los pesos  $w_{i,j}$  se agrupan en la matriz  $\mathbf{W}$  y las entradas y los términos independientes en los vectores  $\mathbf{x}$  y  $\mathbf{b}$  respectivamente, modificando las expresiones 1 y 2:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b} \quad (3)$$

$$\mathbf{y} = \phi(\mathbf{z}) \quad (4)$$

Los *Perceptrons* son en sí arquitecturas muy simples con capacidades limitadas pero pueden encadenarse para aumentar su capacidad de cómputo. Esta arquitectura se llama *Perceptron* multicapa o MLP (*Multilayer Perceptron*). Así, las salidas de la primera capa de neuronas TLU es utilizada como entrada para la segunda capa de TLUs de forma sucesiva, como se observa en la Figura ??.

Denotando como  $\mathbf{h}_l$  la salida de la capa  $l$  en un MLP con  $L$  capas y a los pesos de las neuronas de la misma capa como  $\mathbf{W}_l$  y  $\mathbf{b}_l$ , se modifican las ecuaciones 3 y 4 de la forma:

$$\mathbf{h}_l = \phi(\mathbf{W}_l^T \mathbf{h}_{l-1} + \mathbf{b}_l) \quad (5)$$

$$\mathbf{y} = \mathbf{h}_L \quad (6)$$

Los MLPs pueden ser utilizados en diversas tareas como la clasificación o la regresión. Son capaces incluso de predecir varios valores al mismo tiempo dependiendo del número de TLUs presentes en la última capa de neuronas.

### 3.2. Funciones de activación

Existe una gran multitud de funciones de activación, todas ellas no lineales, para cada tarea en la que se apliquen las neuronas artificiales. Su implementación per-



mite que las neuronas realicen transformaciones no lineales de los datos, siendo así capaces de ajustarse a una mayor variedad de patrones. Sin embargo, las funciones más básicas como la Heaviside no son compatibles con los métodos actuales de entrenamiento de modelos de aprendizaje profundo. Por ello, se propusieron otras funciones más complejas como la función sigmoide  $\sigma(z)$ , la función lineal rectificada  $\text{ReLU}(z)$  (*Rectified Linear Unit*) y la tangente hiperbólica  $\tanh(z)$ .

Las funciones de activación han sufrido un empujón en los últimos años para eliminar las áreas de las funciones como ReLU con gradientes nulos o puntos no derivables. Algunas de ellas son Leaky ReLU y Mish.

### 3.3. Entrenamiento de redes neuronales y *backpropagation*

Como se ha mencionado anteriormente, además de neuronas y funciones de activación, es necesario dotar al modelo de aprendizaje profundo de una métrica que evalúe la precisión de sus resultados y un método matemático capaz de modificar la red neuronal basándose en dicha métrica. Para obtener salidas distintas, la forma más sencilla es modificar ligeramente los valores contenidos en la matriz de pesos  $\mathbf{W}$  de forma aleatoria. Sin embargo, existen protocolos más eficaces como *gradient descent*.

#### 3.3.1. Gradient descent

*Gradient descent* es un algoritmo de entrenamiento que modifica los parámetros que influyen en la salida de un modelo con el objetivo de minimizar una función de coste. Los parámetros de una red neuronal son los pesos  $\mathbf{W}$  incluyendo los términos independientes  $\mathbf{B}$ . La variación introducida en los parámetros está controlada por la tasa de aprendizaje o *learning rate*  $\eta$ .

Con cada cambio en los parámetros, es posible calcular cuánto varía la función de coste a través de su gradiente con respecto a los parámetros modificados  $\nabla_{\mathbf{W}}$ . Un gradiente positivo indica la dirección en la que la función de coste aumenta, por lo que los cambios deberán ir en la dirección opuesta. De esta forma, denominando  $LF$  a la función de coste utilizada, el algoritmo *gradient descent* se resume en:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (7)$$

donde  $\mathbf{W}^{t-1}$  son los parámetros iniciales y  $\mathbf{W}^t$  son los parámetros resultantes.

A lo largo de múltiples iteraciones, *gradient descent* es capaz de obtener los pesos adecuados para cada neurona de tal forma que minimicen la función de coste aplicada. Sin embargo, las funciones de coste presentan valles que podrían confundir al algoritmo *gradient descent* haciéndole creer que se encuentra en el mínimo absoluto. Por ello, es común añadir un término aleatorio  $\alpha$  que permita escapar de los mínimos locales. Añadiéndolo a la Ecuación 7 se consigue la expresión para *stochastic*

*gradient descent*:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \alpha \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (8)$$

Estas variaciones en la expresión matemática que define el algoritmo *gradient descent* son los llamados optimizadores. Existen una gran cantidad de ellos como SDG (*Stochastic Gradient Descent*), *Momentum*, RMSProp o Adam. Cada uno de ellos implementa una mejora de la eficiencia respecto al anterior.

En este trabajo se utiliza un optimizador para aumentar la eficiencia de *gradient descent* propuesto en [10]. Partiendo de la premisa de que las iteraciones de *gradient descent* que producen un error mayor en la función de coste ofrecen más información que aquellas con errores más pequeños, dar un peso mayor a las primeras aumentará la eficacia del algoritmo. Para ello se modifica la Ecuación 7 añadiendo la norma euclídea al cuadrado del error cometido en cada iteración:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \left\| \frac{y - \hat{y}_{t-1}}{y} \right\|^2 \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (9)$$

donde  $y$  es la salida real esperada e  $\hat{y}_{t-1}$  es la salida arrojada por la red neuronal en la iteración  $t - 1$  de *gradient descent*.

### 3.3.2. Funciones de coste

Las funciones de coste son una parte esencial del algoritmo *gradient descent* y no están relacionadas con las funciones de activación. Son el objetivo a minimizar a través de la modificación de los parámetros de la red neuronal. El cambio en los parámetros modifica el procesamiento que realiza la red sobre los datos de entrada y, consecuentemente, en la salida obtenida. La función de coste debe ajustarse al objetivo de la red neuronal para que los resultados sean válidos.

El algoritmo *gradient descent* permite a la red neuronal tomar cualquier atajo para minimizar su función de coste como ignorar ciertos datos clave o inventar nuevos datos. Por ello, ante la gran diversidad de funciones de coste existente a día de hoy, no todas las funciones son implementables para todas las redes neuronales ni para todos los propósitos.

Entre las funciones de coste más populares se encuentran: la entropía cruzada binaria para tareas de clasificación binaria, su variante categórica para clasificación multiclase, similitud del coseno para procesamiento de textos o el error cuadrático medio (MSE) para regresión y procesamiento de series temporales. En este trabajo se utilizará la función de coste MSE (*Mean Squared Error*).

### 3.3.3. *Backpropagation*

El algoritmo *gradient descent* es la base del entrenamiento de redes neuronales pero no es aplicable directamente a redes con múltiples capas de neuronas. En 1970 Seppo Linnainmaa introdujo el algoritmo de diferenciación automática invertida (*reverse-mode automatic differentiation*) que calcula en dos pasadas a toda la red todos los gradientes del error cometido en la salida, medido por la función de coste, respecto a todos y cada uno de los parámetros de la red neuronal. De esta forma se obtiene la contribución al error de cada término independiente  $b_{i,l}$  y cada peso  $w_{i,j,l}$  de cada entrada  $j$  de cada neurona  $i$  de cada capa  $l$ . El algoritmo *gradient descent* puede utilizar estas aportaciones al error para calcular la magnitud en la que cada parámetro debe ser modificado.

El algoritmo *backpropagation*, también llamado retropropagación, es la combinación del algoritmo de Linnainmaa con *gradient descent*. En primera instancia, la red neuronal procesa los datos de entrada para calcular una salida. Este es el pase hacia delante o *forward pass*. Es el funcionamiento normal de una red neuronal salvo que se guardan todas las salidas de todas las neuronas de todas las celdas en vez de desecharlas y dar únicamente la salida final. Con todas esas salidas se ejecuta *reverse-mode automatic differentiation* desde la última capa hasta la primera para calcular la contribución al error perteneciente a cada capa. Este es el pase hacia atrás o *backward pass*. Por último, todos los parámetros son modificados con *gradient descent*.

Es muy importante que los parámetros estén inicializados de forma aleatoria para que se puedan distinguir los unos de los otros. Si no fuera así, el algoritmo *reverse-mode automatic differentiation* sería incapaz de saber qué neurona ha contribuido más o menos al error y *gradient descent* las actualizaría en la misma cuantía. Al ser todas las neuronas iguales, actuarían como una sola con la consecuente reducción de la capacidad de cómputo de la red neuronal.

Por motivos similares no todas las funciones de activación son válidas. Aunque la función de coste indica el error final cometido en la salida, *backpropagation* debe calcular la contribución de cada parámetro al total. Cambiar un peso  $w_{i,j,l}$  o un término independiente cualquiera  $b_{i,l}$  de cualquier neurona no supone un cambio directo en la salida de dicha neurona sino que ha de pasar por la función de activación. Por lo tanto, la función de activación debe ser diferenciable para que *backpropagation* pueda calcular cuánto ha de modificar el peso o término término independiente deseado para mover la salida final en una dirección determinada.

Por esta razón, todas las funciones no diferenciables utilizan trucos para sustituir sus puntos no diferenciables con otros valores. Sin embargo, las funciones de activación más problemáticas son aquellas con gradientes nulos puesto que anulan el efecto de *gradient descent*. Un gran ejemplo de gradientes nulos es la función Heaviside( $z$ ), que tiene gradiente cero para todo  $z$  salvo  $z = 0$  donde no es diferenciable.

La función de activación utilizada en este trabajo y en la gran mayoría de redes neuronales es ReLU( $z$ ). Tiene un gradiente no nulo para  $z > 0$ , un punto no diferenciable en  $z = 0$  y gradiente nulo para  $z < 0$ . Esto implica que, si la combinación

de entradas y pesos de la neurona es menor a cero, la neurona queda inutilizada. Es una gran ventaja que permite a la red neuronal eliminar neuronas innecesarias para cada dato de entrada.

La elección de ReLU como función de activación no sólo recae en su capacidad para generar gradientes no nulos para  $z > 0$ . Los componentes electrónicos sobre los que se realizan las operaciones típicas de las redes neuronales tienen limitaciones. Funciones de activación muy complejas requieren grandes inversiones en capacidad de cómputo. Además, a nivel de *software* y librerías de programación, la función ReLU está altamente optimizada para obtener el máximo rendimiento de la electrónica.

Las redes neuronales basadas en *online learning* ejecutan el algoritmo *backpropagation* en cada entrenamiento incremental. La precisión de las salidas es evaluada constantemente y los pesos de las capas de neuronas son modificados consecuentemente para minimizar la función de coste.

### 3.4. Selección de modelos

Los modelos de redes neuronales son entrenados aplicando *backpropagation* hasta que los pesos de la red son los idóneos para minimizar la función de coste. Sin embargo, los pesos no son los únicos parámetros que caracterizan a la red neuronal evaluada. Hay una gran multitud de variables, llamadas hiperparámetros, que influyen en los resultados obtenidos. Un conjunto de hiperparámetros con unos valores concretos identifica a un modelo.

Existen diferentes estrategias en la búsqueda de los mejores hiperparámetros. Entre las estrategias más comunes se encuentran el *grid search* y el *random search*. La primera itera sobre una matriz de valores concretos establecidos mientras la segunda utiliza conjuntos de valores obtenidos de forma aleatoria dentro de un rango delimitado. *Grid search* ofrece un gran control sobre los valores utilizados en la búsqueda mientras *random search* ofrece la posibilidad de encontrar buenos conjuntos de valores en un rango de búsqueda más amplio. Ninguna estrategia es infalible, siempre cabe la posibilidad de que el modelo para el que se realiza la búsqueda no sea un buen modelo [12].

Según cómo se diseñe el programa o modelo de aprendizaje automático y el valor de sus hiperparámetros, su rendimiento a la hora de obtener el patrón subyacente de unos datos puede variar mucho. No existe *a priori* un modelo mejor que otro [13]. Por ello, se ha diseñado un protocolo de selección de modelos de aprendizaje automático.

El aprendizaje automático se divide en tres fases que contienen 3 sets de datos diferentes: entrenamiento, validación y test. El entrenamiento refiere a la primera fase, donde el programa o modelo aprende en la medida que le es posible el patrón subyacente presente en los datos de entrenamiento a través del algoritmo *backpropagation*. El grado de aprendizaje correcto del patrón de los datos de entrenamiento se denomina error de entrenamiento o  $E_{in}$ , siendo 0 el mejor valor posible.

El modelo, una vez pasada la fase de entrenamiento, dejará de aprender para con-

vertirse en un modelo en producción que se dedicará a aplicar el patrón aprendido en el entrenamiento a nuevas muestras. Sin embargo, aunque el modelo tenga un  $E_{in}$  bajo, puede no dar las respuestas correctas a las nuevas muestras. El error cometido en las muestras en producción se denomina como  $E_{out}$  y el error de generalización se calcula como  $E_{out} - E_{in}$ . Para evitar que esto ocurra y se lance un modelo a producción que no funcione bien, provocando en la mayoría de casos pérdidas económicas y oportunidades de negocio, se introducen las fases de validación y test.

No se pueden comparar modelos entre sí directamente usando  $E_{in}$  puesto que esto sólo conllevaría a la elección del modelo con menor  $E_{in}$  y, en consecuencia, con mayor  $E_{out}$ . Esta situación es conocida como *overfitting*. La fase de validación se encarga de comparar los diferentes modelos propuestos haciéndoles procesar un set de datos de validación como si estuvieran en producción. El desempeño de los modelos en esta fase o  $E_{val}$  sí que puede utilizarse para comparar modelos.

Aunque existen múltiples estrategias de validación como *Grid Search*, *Random Search*, *k-fold*... no todas son aplicables a la predicción de series temporales. Los datos de validación siempre han de ser posteriores a los datos de entrenamiento y los datos de test siempre han de ser posteriores a los datos de validación [14]. Por lo tanto, estrategias como *k-fold validation* quedan descartadas de este trabajo.

La fase de validación tiene el mismo problema que la fase de entrenamiento. Cabe la posibilidad de que el modelo escogido haya realizado *overfitting* en los datos de validación y, aunque tenga un valor pequeño de  $E_{val}$ , tenga un gran error  $E_{out}$ . Por ello, la fase de test reentrena únicamente el mejor modelo de la fase de validación utilizando los datos de entrenamiento y validación y mide el nuevo rendimiento con un set de datos de test. El error  $E_{test}$  es entonces una buena estimación de  $E_{out}$ .

Es importante recalcar que los datos de test sólo deben ser utilizados en la fase de test. Utilizarlos en otras fases como la de entrenamiento o en el reentrenamiento del modelo tras la validación provocará una distorsión en la medida  $E_{test}$  llamada *data leakage*. Tampoco ha de usarse para comparar modelos como si fuera un test de validación [3, 15, 16].

### 3.5. Preprocesado de los datos de entrada

Aunque la elección de un buen modelo con los hiperparámetros correctos es muy importante, también lo es la disponibilidad de datos veraces. Los datos utilizados tanto para el entrenamiento como la validación o el test han de ser: suficientemente numerosos como para permitir al algoritmo *backpropagation* llegar al mínimo de la función de coste y representativos del patrón subyacente que se desea estimar.

Las características de los datos de entrada son altamente influyentes en el rendimiento de los modelos de aprendizaje automático. Por ello se ha de actuar sobre los datos para evitar errores en su procesamiento. Las acciones más comunes son: imputación y normalización. La imputación se encarga de los posibles huecos en los datos, rellenándolos con ceros u otro valor significativo para no interrumpir el patrón subyacente a estimar. La normalización aplica una transformación a los datos para desplazarlos

y escalarlos en el rango  $[0, 1]$ .

La normalización es especialmente útil para evitar que los modelos ignoren los datos más pequeños en favor de los datos más grandes. Sin embargo, la normalización se aplica respecto a los datos incluidos únicamente en el set de entrenamiento. El resto de los datos, tanto del set de validación como de test, son transformados utilizando los valores mínimo  $min_{train}$  y máximo  $max_{train}$  del set de entrenamiento:

$$x_{norm} = \frac{x - min_{train}}{max_{train} - min_{train}} \quad (10)$$

De cualquier otra forma, el modelo tendría acceso a los datos del conjunto de validación o test en la fase de entrenamiento. El modelo se adaptaría erróneamente a unos datos no representativos en un proceso llamado *data leakage*.

### 3.6. Técnicas de regularización (dropout, gradient clipping, l1, l2, overfitting, vanishing gradients...)

## 4. Predicción de series temporales

Una serie temporal puede ser representada como un vector en cuyas posiciones se encuentran muestras secuenciales y equiespaciadas en el tiempo de una misma variable. Existen multitud de métodos de predicción de series de datos basados en la extrapolación. Sin embargo, no son específicos de series temporales sino que se aplican a conjuntos de datos genéricos.

El estudio clásico de las series temporales considera que su comportamiento es fruto de la tendencia, las variaciones cíclicas y estacionales y un componente aleatorio. La tendencia es la componente que refleja la evolución lineal a largo plazo de la serie temporal de forma lineal, exponencial... La componente cíclica recoge las oscilaciones periódicas de amplitud superior a un año. Por último, la componente estacional recoge las oscilaciones periódicas de amplitud inferiores a un año como las oscilaciones mensuales, semanales, diarias... Estas tendencias pueden generar una combinación aditiva si son independientes entre sí o multiplicativa si están correlacionadas entre sí [17].

En la actualidad es mucho más común recurrir a modelos de aprendizaje automático para las tareas de predicción. La mayoría de estos modelos se basan en arquitecturas neuronales recurrentes, capaces de memorizar los aspectos temporales más importantes de la serie analizada.

## 4.1. Modelos autorregresivos de medias móviles

Como su propio nombre indica, los modelos autorregresivos de medias móviles para la predicción de series temporales están basados en los modelos de medias móviles y autorregresivos. Las medias móviles son una manera simple de obtener una predicción cuando no hay una tendencia marcada ni tampoco estacionalidad. Se utiliza el término medias móviles puesto que conforme se va disponiendo de nuevas observaciones se pueden calcular nuevas medias mediante la inclusión de las nuevas muestras y el desprendimiento de las más antiguas [18]. De esta forma se eliminan las oscilaciones de mayor frecuencia, actuando como un filtro paso bajo.

Las medias móviles unilaterales permiten estimar el valor futuro de la serie temporal  $\hat{y}_t$  como la media de las  $m$  muestras inmediatamente anteriores:

$$\hat{y}_t = \frac{1}{m} * \sum_{i=1}^m y_{t-i}$$

Asímismo, la media móvil puede combinarse con una ponderación  $k_i$ . La ponderación más habitual es la exponencial asumiendo que las muestras más recientes contribuyen más a la correcta predicción que las muestras más antiguas:

$$\hat{y}_t = \frac{1}{m} * \sum_{i=1}^m y_{t-i} * k_{t-i}$$

El modelo ARMA (*AutoRegressive Moving Average*) es el modelo más sencillo que combina las medias móviles y la autorregresión. Utiliza tanto las muestras reales de la serie temporal  $Y_t$  como los errores cometidos en predicciones pasadas  $\epsilon_t$ . Concretamente, la cantidad de muestras y errores cometidos que son tomados en cuenta está estipulada por los parámetros  $p$  y  $q$  respectivamente. Siendo  $\alpha_t$  y  $\theta_t$  la ponderación aplicada a cada muestra y error [15, 19, 20]:

$$\begin{aligned}\hat{y}_t &= \sum_{i=1}^p \alpha_i * y_{t-i} + \sum_{i=1}^q \theta_i * \epsilon_{t-i} \\ \epsilon_t &= y_t - \hat{y}_t\end{aligned}$$

El primer sumatorio es la parte autorregresiva, realiza una regresión basada en datos pasados. El segundo sumatorio es la media móvil de los errores cometidos en predicciones pasadas. Este modelo se denomina ARMA (p,q).

ARMA fue desarrollada inicialmente para la predicción de datos financieros [21] pero ha dado grandes resultados en sus adaptaciones a la predicción de tráfico en red a corto plazo [22, 23]. Sin embargo, el modelo ARMA presupone que la serie

temporal es estacionaria, es decir, que sus propiedades estadísticas son constantes en el tiempo, sin tendencias ni comportamientos periódicos [15].

El modelo ARIMA (*AutoRegressive Integrated Moving Average*) [24] introduce la distancia entre dos muestras como instrumento para eliminar las tendencias presentes en la serie temporal. Al diferenciar el valor de una muestra al valor de la siguiente se reduce un orden de la tendencia. Aplicar una segunda diferenciación se reduce otro orden más, etc. De esta forma, la serie temporal resultante tras  $d$  órdenes de diferenciación es estacionaria y se puede aplicar el modelo ARMA. A este proceso se le denomina ARIMA( $p,d,q$ ).

A su vez, el modelo ARIMA falla en analizar las oscilaciones periódicas que puedan estar presentes en los datos. El modelo SARIMA (*seasonal ARIMA*) mejora el modelo ARIMA añadiendo un procesado en paralelo de la componente periódica para una frecuencia determinada. Este procesado adicional también es realizado a partir de un modelo ARIMA con sus propios parámetros  $P, D, Q$  imitando los parámetros  $p, d, q$ . Así, la notación de este último modelo es SARIMA( $p,d,q$ )( $P,D,Q,s$ ) siendo  $s$  el periodo de la componente frecuencial analizada.

## 4.2. Modelos de aprendizaje profundo aplicados a la predicción de series temporales

En los últimos años, los modelos autorregresivos descritos anteriormente han sido relevados por modelos de aprendizaje automático y, más concretamente, aprendizaje profundo. Éstos últimos son especialmente populares gracias a su capacidad de extraer el patrón subyacente de los datos sin necesidad de intervención humana para adaptar los algoritmos a las series temporales. El *deep learning* elimina la necesidad de analizar los datos en busca de características estacionarias, tendencias, componentes periódicas, anomalías... Un mismo modelo de aprendizaje profundo es capaz de adaptarse por sí mismo a cualquier serie temporal, lo que no disminuye la importancia de las estrategias de entrenamiento y selección de modelos como la normalización de los datos o la validación.

Las redes neuronales han evolucionado desde su creación para mejorar su rendimiento en los diferentes escenarios en los que se aplican, aunque no son exclusivas de ellos. Por ejemplo, el *Perceptron* dio lugar a las neuronas convolucionales para tareas de reconocimiento visual. En el caso de las series temporales, las redes neuronales evolucionaron hacia las neuronas recurrentes que forman las redes neuronales recurrentes o RNNs (*Recurrent Neural Networks*).

Las RNNs son utilizadas en todo tipo de secuencias temporales de cualquier naturaleza como muestras de audio, registros históricos numéricos e incluso documentos escritos. Para realizar sus predicciones, al igual que los modelos clásicos autorregresivos, las RNNs y todos los modelos de aprendizaje automático asumen en sus predicciones que el patrón subyacente presente en los datos de entrenamiento se perpetúa en los datos futuros.



#### 4.2.1. Redes neuronales recurrentes

Las redes neuronales recurrentes o RNNs (*Recurrent Neural Networks*), a diferencia de las TLUs, no están basadas en ninguna estructura biológica en concreto. Son capas de neuronas capaces de retener información de muestras procesadas con anterioridad. En cada momento  $t$ , las neuronas recurrentes más simples, además de procesar las entradas  $\mathbf{x}_t$  actuales, procesan su propia salida anterior  $\hat{y}_{t-1}$  con su propio peso  $w_{\hat{y}}$  para generar la salida actual  $\hat{y}_t$ . Esto hace a las RNN ideales para el procesamiento de secuencias de datos como, por ejemplo, el histórico de tráfico de telecomunicaciones de una ciudad concreta.

$$\hat{y}_t = \phi(\mathbf{W}_x^T \mathbf{x}_t + b + w_{\hat{y}} * \hat{y}_{t-1}) \quad (11)$$

Al igual que las TLUs, las neuronas recurrentes pueden agruparse para formar capas. Modificando la ecuación 11 con la forma matricial de los pesos  $\mathbf{W}_x$ ,  $\mathbf{W}_{\hat{y}}$  y la forma vectorial de las salidas  $\hat{\mathbf{y}}_t$ , la salida de una capa de neuronas recurrentes se describe como:

$$\hat{\mathbf{y}}_t = \phi(\mathbf{W}_x^T \mathbf{x}_t + \mathbf{W}_{\hat{y}}^T \hat{\mathbf{y}}_{t-1} + \mathbf{b}) \quad (12)$$

donde  $\phi$  sigue siendo la función de activación presente en la neurona. Gracias al algoritmo *gradient descent*, las capas de neuronas recurrentes son capaces de adaptarse al patrón subyacente de las series temporales sin necesidad de intervención humana.

Es importante recalcar la naturaleza de  $\mathbf{W}_x$ ,  $\mathbf{W}_{\hat{y}}$  y  $\mathbf{b}$ . A diferencia de los vectores de entradas y salidas, los pesos de las neuronas recurrentes son los mismos independientemente del momento  $t$ , al igual que los pesos de las TLUs y los MLPs. Esto permite tratar a una misma secuencia como un sólo dato, manteniendo la integridad de las capas de neuronas a lo largo de toda la serie temporal.

La similitud con las TLUs y los *Perceptrons* continúa al poder agrupar varias capas de neuronas recurrentes de forma secuencial como un MLP. Las salidas  $\hat{\mathbf{y}}_t$  de las capas inferiores conforman las entradas  $\mathbf{x}_t$  de las capas superiores, pudiendo aplicar el algoritmo *backpropagation* para encontrar los valores idóneos de  $\mathbf{W}_x$ ,  $\mathbf{W}_{\hat{y}}$  y  $\mathbf{b}$ .

La retroalimentación de la salida  $\hat{y}$  en una neurona recurrente es el mecanismo que otorga la capacidad de memoria, así como lo hacía la autorregresión en los modelos clásicos. Las RNNs procesan secuencias de datos mientras mantienen la información de las muestras pasadas en el llamado estado oculto  $h_t$ . Este estado puede ser directamente la salida pasada  $h_t = \hat{y}_{t-1}$  o una transformación más compleja de la misma.

#### 4.2.2. Minibatch con las series temporales

#### 4.2.3. Inconvenientes de las neuronas recurrentes

Al igual que las TLUs, las neuronas recurrentes son estructuras muy simples por lo que su rendimiento está acotado por el ámbito en el que se utilizan. Concretamente, las RNNs sufren varios problemas al enfrentar secuencias de larga duración.

El primer problema recae en la inestabilidad de la retroalimentación. Como los pesos  $\mathbf{W}_x$ ,  $\mathbf{W}_{\hat{y}}$  y  $\mathbf{b}$  son constantes a lo largo de toda la serie temporal, se aplican tantas veces como instantes temporales tenga la secuencia procesada. La retroalimentación puede provocar que las salidas disminuyan hasta anularse o, con una función de activación no saturante como ReLU, aumenten sin control hasta inutilizar toda la red.

Algunos mecanismos para evitar la inestabilidad incluyen funciones de activación que saturan la salida como la tangente hiperbólica, una tasa de aprendizaje  $\eta$  más pequeña o *gradient clipping*.

El segundo problema reside en, debido a la simpleza de las neuronas recurrentes, su limitada capacidad de memoria. A medida que la red neuronal procesa secuencias  $\mathbf{x}_t$ , los pesos  $\mathbf{W}_{\hat{y}}$  modifican el estado  $h_t$  eliminando la información más antigua en favor de la información más reciente.

Esto supone un gran problema puesto que la información más reciente no siempre es útil para estimar el patrón subyacente de la serie temporal. Sin un mecanismo de decisión, el estado oculto  $h_t$  es extremadamente vulnerable a cualquier anomalía o dato irrelevante presente en la secuencia procesada. Para resolver este problema se introdujo la neurona LSTM en 1997.

#### 4.2.4. LSTM

### 5. Estructuras avanzadas de *Deep Learning* para la predicción de series temporales

#### 5.1. CNNs

##### 5.1.1. Filtros, convoluciones, neuronas, ecuaciones...

#### 5.2. CNN+LSTM

##### 5.2.1. Neuronas, conexiones, ecuaciones, flujo de datos...

## Referencias

- [1] G. Bell, “Bell’s law for the birth and death of computer classes,” *Communications of the ACM*, vol. 51, no. 1, pp. 86–94, Jan 2008. [Online]. Available: <https://doi.org/10.1145/1327452.1327453>
- [2] A. Azzouni and G. Pujolle, “A long short-term memory recurrent neural network framework for network traffic matrix prediction,” *arXiv preprint arXiv:1705.05690*, Jun 2017. [Online]. Available: <http://arxiv.org/abs/1705.05690>
- [3] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H.-T. Lin, *Learning from Data: A Short Course*. United States: AMLBook, 2012.
- [4] N. Bui, M. Cesana, S. A. Hosseini, Q. Liao, I. Malanchini, and J. Widmer, “A survey of anticipatory mobile networking: context-based classification, prediction methodologies, and optimization techniques,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1790–1821, 2017.
- [5] T. Guo, Z. Xu, X. Yao, H. Chen, K. Aberer, and K. Funaya, “Robust online time series prediction with recurrent neural networks,” in *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016, pp. 816–825.
- [6] Y. Duan, G. Fu, N. Zhou, X. Sun, N. C. Narendra, and B. Hu, “Everything as a service (xaas) on the cloud: Origins, current and future trends,” pp. 621–628, 2015.
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under Concept Drift: A Review,” *IEEE Transactions on Knowledge & Data Engineering*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TKDE.2018.2876857>
- [8] G. Widmer and M. Kubat, “Learning in the presence of concept drift and hidden contexts,” *Machine Learning*, vol. 23, no. 1, pp. 69–101, 1996. [Online]. Available: <https://doi.org/10.1007/BF00116900>
- [9] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, Mar 2014. [Online]. Available: <https://dl.acm.org/doi/epdf/10.1145/2523813>
- [10] P. Casaseca-de-la Higuera, J. M. Aguiar, P. Amado-Caballero, A. J. Morgado, J. d. S. Moreira, C. Luo, and X. Wang, “Online active learning for LSTM-based network traffic prediction,” 2023, iEEE (Unpublished Draft / Preprint).
- [11] A. Román Antolín, “Predicción de tráfico de red usando arquitecturas neuronales avanzadas,” Trabajo Fin de Grado, Universidad de Valladolid, Valladolid, España, Sep 2025.
- [12] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: <http://jmlr.org/papers/v13/bergstra12a.html>

- [13] D. H. Wolpert, “The lack of a priori distinctions between learning algorithms,” *Neural Computation*, vol. 8, no. 7, pp. 1341–1390, 10 1996. [Online]. Available: <https://doi.org/10.1162/neco.1996.8.7.1341>
- [14] F. Chollet, *Deep learning with Python*, 2nd ed. Madrid: Anaya Multimedia, 2020.
- [15] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems*, 3rd ed. Beijing: O’Reilly, 2022.
- [16] L. N. Smith, “A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay,” *CoRR*, vol. abs/1803.09820, 2018. [Online]. Available: <http://arxiv.org/abs/1803.09820>
- [17] M. González and I. M. del Puerto, *Series Temporales*, ser. Colección manuales uex. Cáceres, España: Universidad de Extremadura, Servicio de Publicaciones, 2009, vol. 60.
- [18] J. C. García Díaz, *Predicción en el dominio del tiempo: análisis de series temporales para ingenieros*, ser. Manual de referencia. Valencia: Editorial de la Universidad Politécnica de Valencia, 2016.
- [19] P. Hoong, “Bittorrent network traffic forecasting with arma,” *International Journal of Computer Networks and Communications*, vol. 4, pp. 143–156, 07 2012.
- [20] H. L. S., “A study in the analysis of stationary time series. by herman wold. [pp. 214 + viii. almqvist and wiksell’s boktryckeri-a.-b., uppsala. 1938. price kr. 6.],” *Journal of the Institute of Actuaries*, vol. 70, no. 1, p. 113–115, 1939.
- [21] K. H. Poo, I. K. T. Tan, and K. H. Ng, “Computing autoregressive moving average (ARMA) with CRONOS: A case study of Bursa Malaysia,” in *Proc. 2nd Int. Conf. Eng. ICT (ICEI)*, Melaka, Malaysia, Feb 2010, pp. 516–520.
- [22] M. Papadopouli, H. Shen, E. Raftopoulos, M. Ploumidis, and F. Hernandez-Campos, “Short-term traffic forecasting in a campus-wide wireless network,” in *2005 IEEE 16th International Symposium on Personal, Indoor and Mobile Radio Communications*, vol. 3, 2005, pp. 1446–1452.
- [23] S. Jung, C. Kim, and Y. Chung, “A prediction method of network traffic using time series models,” vol. 3982, 11 2006, pp. 234–243.
- [24] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 1970. [Online]. Available: <https://onlinelibrary.wiley.com/doi/book/10.1002/9781118619193>